Enrollment no:240840116048

Practical-7 Develop advanced visualization (Box plot,Heatmap,Pair plot,Violine plot)

TASK 1 CREATE AND INSPECT THE DATA

Step 1.1 Import require libraries

```
# Import Pandas for data manipulation and analysis
import pandas as pd
# Import Matplotlib for creating graphs and visualizations
import matplotlib.pyplot as plt
# Import Seaborn for statistical data visualization
import seaborn as sns
```

Step 1.2 Create a Dataset

```
# Create a Dataset
data = {
    "Student_Id":[1,2,3,4,5,6,7,8,9,10,11,12,13,14],
    "Study_Hours":[1.5,2.0,2.5,3.0,3.5,4.0,4.5,5.0,5.5,6.0,6.5,7.0,8.0,9.0],
    "Attendance":[65,70,72,75,78,80,82,85,87,90,91,93,95,97],
    "Assignment_score":[55,60,62,65,68,72,75,78,80,84,87,90,94,98],
    "Midterm_marks":[48,52,55,58,61,65,68,72,75,79,82,86,91,96],
    "Final_marks":[45,50,52,57,62,68,72,76,81,85,89,93,97,100]
}

# Display the Dataset
df = pd.DataFrame(data)
df
```

	Student_Id	Study_Hours	Attendance	Assignment_score	Midterm_marks	Final_marks
0	1	1.5	65	55	48	45
1	2	2.0	70	60	52	50
2	3	2.5	72	62	55	52
3	4	3.0	75	65	58	57
4	5	3.5	78	68	61	62
5	6	4.0	80	72	65	68
6	7	4.5	82	75	68	72
7	8	5.0	85	78	72	76
8	9	5.5	87	80	75	81
9	10	6.0	90	84	79	85
10	11	6.5	91	87	82	89
11	12	7.0	93	90	86	93
12	13	8.0	95	94	91	97
13	14	9.0	97	98	96	100

Next steps: Generate code with df | New interactive sheet

```
# Display the First Five records
df.head()
```

	Student_Id	Study_Hours	Attendance	Assignment_score	Midterm_marks	Final_marks
0	1	1.5	65	55	48	45
1	2	2.0	70	60	52	50
2	3	2.5	72	62	55	52
3	4	3.0	75	65	58	57
4	5	3.5	78	68	61	62

Next steps: Generate code with df | New interactive sheet

```
#Display information about coulum and data types
df.info()
```

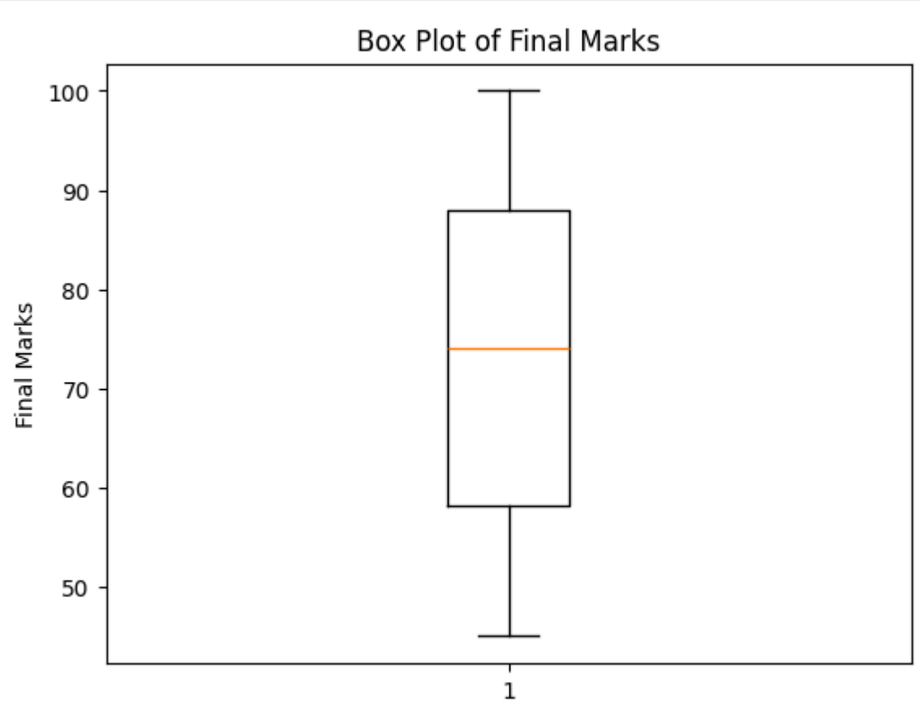
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 14 entries, 0 to 13
Data columns (total 6 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   Student_Id        14 non-null     int64
 1   Study_Hours       14 non-null     float64
 2   Attendance        14 non-null     int64
 3   Assignment_score  14 non-null     int64
 4   Midterm_marks     14 non-null     int64
 5   Final_marks       14 non-null     int64
dtypes: float64(1), int64(5)
memory usage: 804.0 bytes
```

```
#Display basic statistical information
df.describe()
```

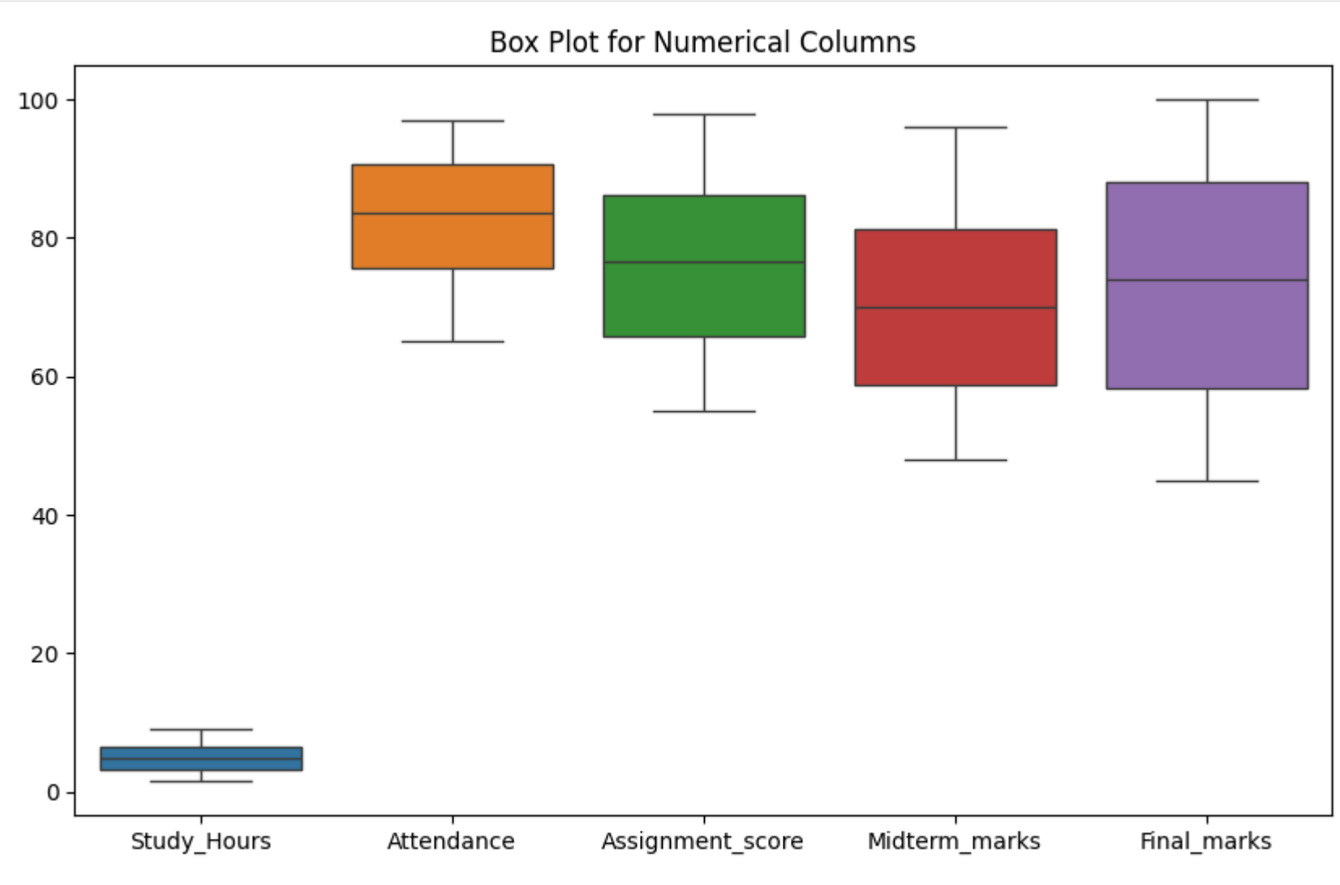
	Student_Id	Study_Hours	Attendance	Assignment_score	Midterm_marks	Final_marks
count	14.0000	14.000000	14.000000	14.000000	14.000000	14.000000
mean	7.5000	4.857143	82.857143	76.285714	70.571429	73.357143
std	4.1833	2.273836	9.944903	13.309503	14.960020	18.257168
min	1.0000	1.500000	65.000000	55.000000	48.000000	45.000000
25%	4.2500	3.125000	75.750000	65.750000	58.750000	58.250000
50%	7.5000	4.750000	83.500000	76.500000	70.000000	74.000000
75%	10.7500	6.375000	90.750000	86.250000	81.250000	88.000000
max	14.0000	9.000000	97.000000	98.000000	96.000000	100.000000

TASK 2 CREATE A BOX PLOT

```
# create Box plot for final marks
plt.boxplot(df['Final_marks'])
plt.title('Box Plot of Final Marks')
plt.ylabel('Final Marks')
plt.show()
```




```
# create box plots for  multiple numerical columns
numerical_columns = ['Study_Hours', 'Attendance', 'Assignment_score',
                     'Midterm_marks', 'Final_marks']
plt.figure(figsize=(10, 6))
sns.boxplot(data=df[numerical_columns])
plt.title('Box Plot for Numerical Columns')
plt.show()
```



TASK 3 CREATE A HEATMAP

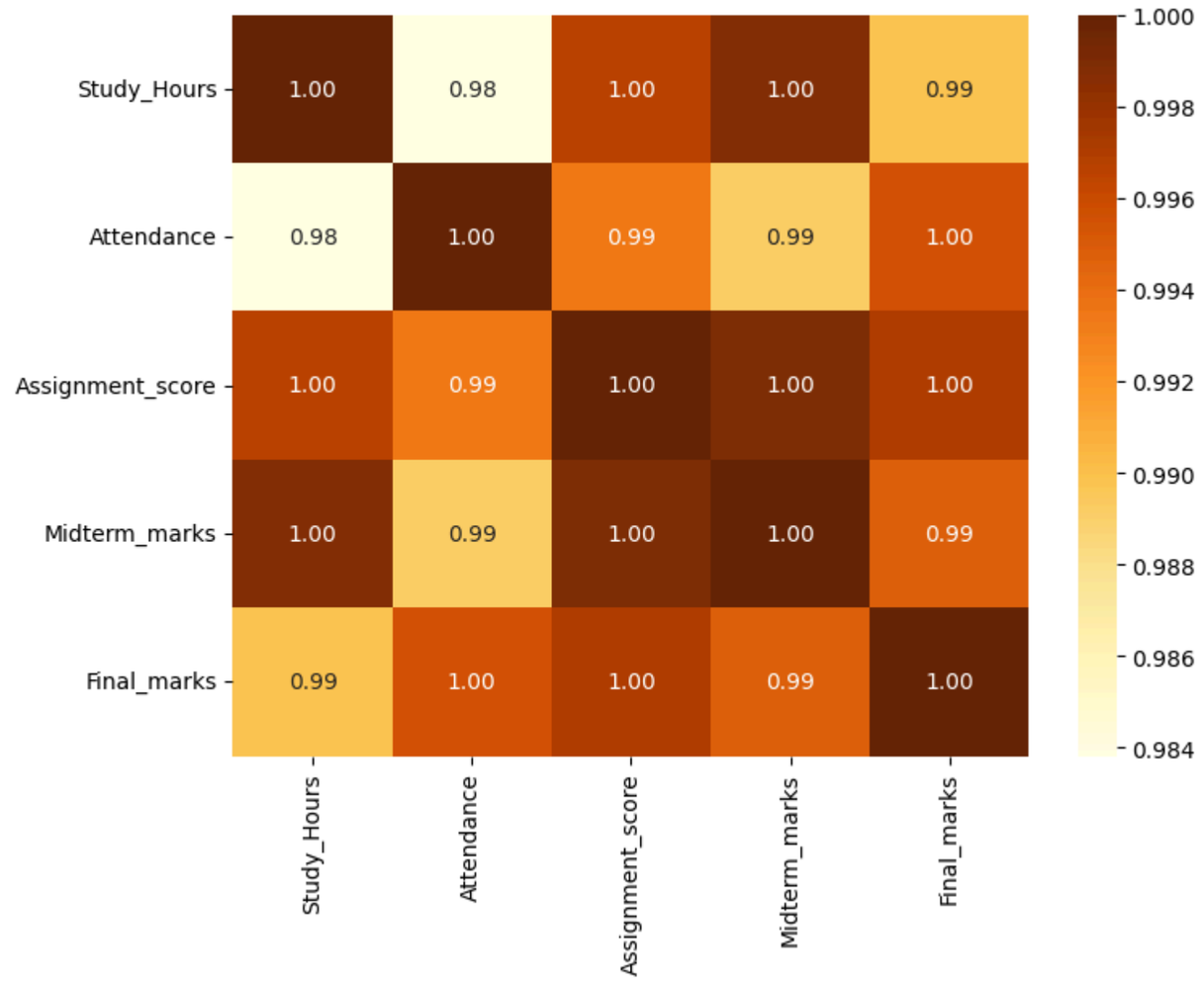
```
# Calculate the correlation matrix
numerical_columns = ['Study_Hours', 'Attendance', 'Assignment_score', 'Midterm_marks', 'Final_marks']
# create a correlation matrix
correlation_matrix = df[numerical_columns].corr()
correlation_matrix
```

	Study_Hours	Attendance	Assignment_score	Midterm_marks	Final_marks
Study_Hours	1.000000	0.983822	0.996554	0.998703	0.989873
Attendance	0.983822	1.000000	0.993531	0.989171	0.995492
Assignment_score	0.996554	0.993531	1.000000	0.998949	0.997041
Midterm_marks	0.998703	0.989171	0.998949	1.000000	0.994785
Final_marks	0.989873	0.995492	0.997041	0.994785	1.000000

Next steps: Generate code with `correlation_matrix` New interactive sheet

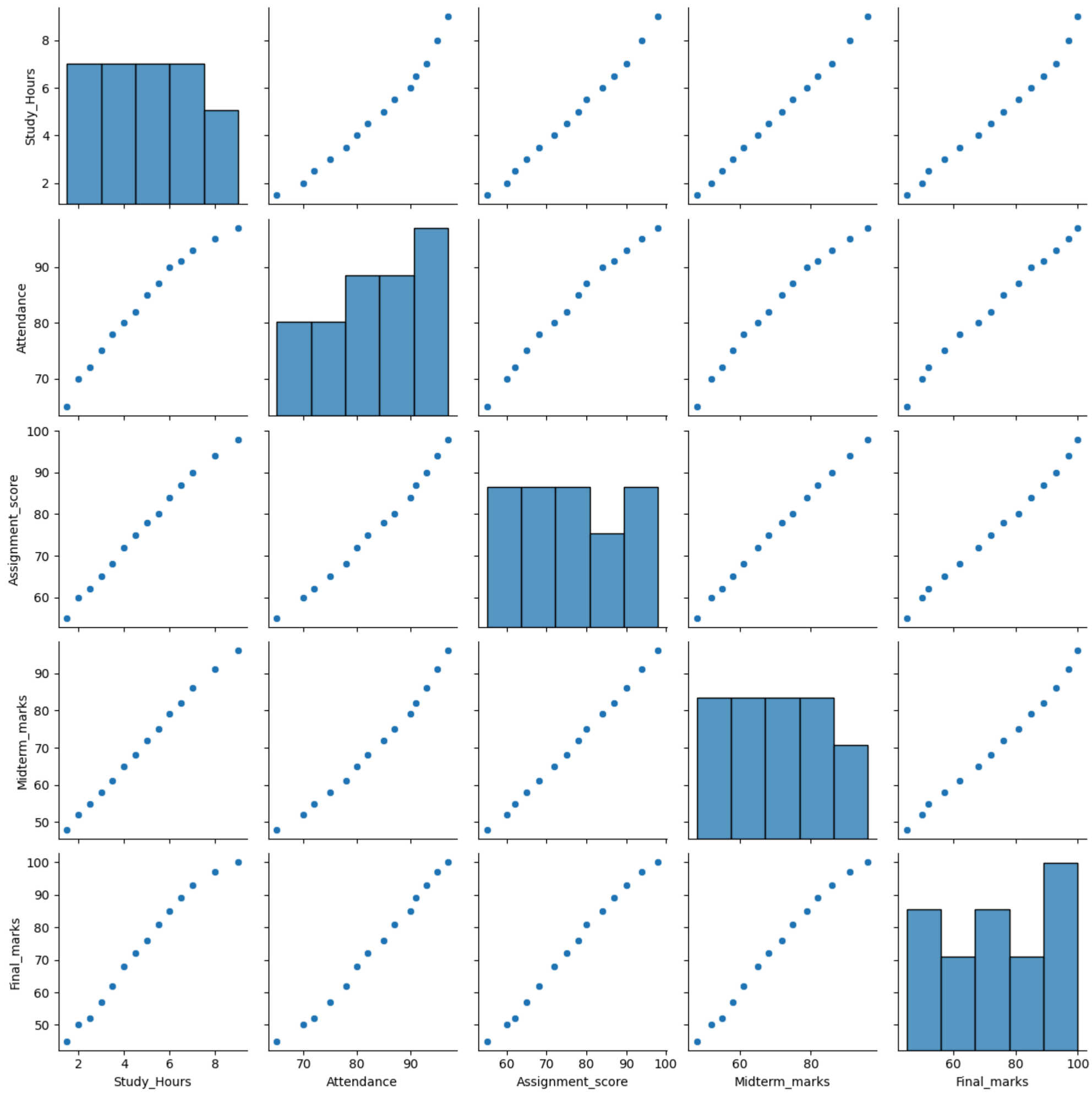
```
# Select numerical columns for correlation analysis
correlation_matrix = df[numerical_columns].corr()
# Create a heatmap
```

```
plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap='YlOrBr', fmt=".2f")
plt.show()
```



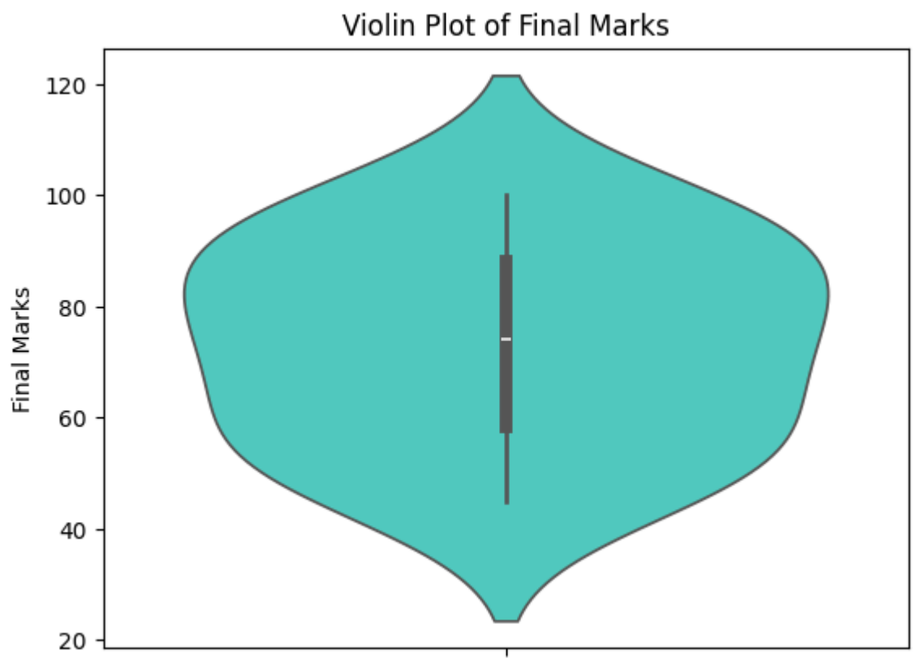
TASK 4 CREAT A PAIR POLT

```
# Create the pair plot
# select the numerical columns that we want to compare
numerical_columns = ['Study_Hours', 'Attendance', 'Assignment_score', 'Midterm_marks', 'Final_marks']
sns.pairplot(df[numerical_columns])
plt.show()
```

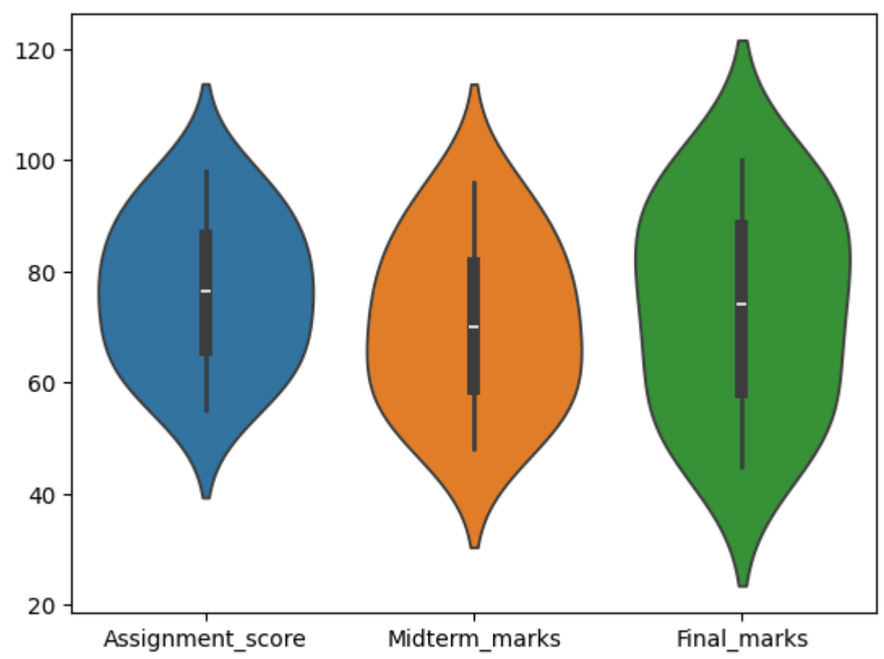


TASK 5 CREATE A VIOLINE PLOT

```
# Create a violine plot for Final Marks
sns.violinplot(data=df, y='Final_marks', color='turquoise' )
plt.title('Violin Plot of Final Marks')
plt.ylabel('Final Marks')
plt.show()
```




```
# Create violine plots for Assignment, midterm and Final marks.
numerical_columns = ['Assignment_score', 'Midterm_marks', 'Final_marks']
sns.violinplot(data=df[numerical_columns])
plt.show()
```



TASK 6 SAVE THE DATASET

```
# Save the student performance dataset as a CSV file

# Save DataFrame to a CSV file
df.to_csv('student_data.csv', index=False)

# Display confirmation message
print('Dataset saved to student_data.csv')
```

```
Dataset saved to student_data.csv
```

```
# Download CSV file from Google Colab
from google.colab import files
files.download('student_data.csv')
print ("Your Data has been Saved 👍")
```

```
Your Data has been Saved 👍
```